CASA Evidence — MFA Enforcement on Administrative Interfaces

Question: *Provide evidence demonstrating that any application exposed administrative interfaces enforce multi-factor authentication for all accounts.*

Application: Provenance (Next.js App Router), using Supabase Auth (GoTrue) TOTP-based multi-factor authentication and Authenticator Assurance Level (AAL) step-up.

---

1. Summary

Every administrative surface in this application — every page under `/admin`, every `/admin` server action, and every `/api/admin/*` route — is required to pass through one of four shared helper functions in `src/lib/admin.ts` (`requireAdmin`, `requireAdminApi`, `requireAdminUser`, `requireAdminUserId`). All four share a single MFA-assurance check (`evaluateAdminMfa`) that enforces, for **every** admin account with no exception:

1. If the account has enrolled an MFA factor but the current session has not completed step-up verification (AAL1 while AAL2 factors exist) → **hard block**, redirected/rejected until the user completes `/auth/verify`.
1. If the account has **no** MFA factor enrolled → allowed through only until a **7-day enrollment grace deadline** stored in the database (`accounts.admin_mfa_grace_deadline`, set automatically by a database trigger the instant an account becomes admin); a shrinking-countdown banner is shown the whole time.
1. Once that grace deadline passes with still no factor enrolled → **hard block** on every subsequent request, page load, action, and API call, until MFA is enrolled.

1 Any error evaluating the account's AAL **fails closed** as "step-up required" rather than silently letting the request through.

There is no code path in `/admin` or `/api/admin` that reaches application logic without first passing through this check — this is demonstrated in §3 by enumerating every admin route/page/action in the codebase and showing each is gated.

---

2. The shared enforcement logic

`evaluateAdminMfa()` is the single implementation all four `require*` helpers call — there is exactly one place in the codebase that decides whether an admin session is MFA-compliant, so the policy cannot drift between different admin surfaces:

```
type MfaOutcome =
  | { kind: 'ok'; requiresMfaSetup: boolean; mfaGraceDeadline: Date | null }
  | { kind: 'step_up_required' }
  | { kind: 'grace_expired' };

/**
 * Shared MFA assurance check used by every require* variant below.
 *
 * - Enrolled factors, session not stepped up (aal1, aal2 required) → 'step_up_required'.
 * - No factors enrolled, still inside the 7-day grace period from
 *   admin_mfa_grace_deadline → 'ok' with requiresMfaSetup=true (banner nudge).
 * - No factors enrolled, grace period expired (or was never set — fail
 *   closed) → 'grace_expired' (CASA 3.3: admin interfaces must use MFA).
 *
 * Any error checking AAL fails closed as 'step_up_required'.
 */
async function evaluateAdminMfa(state: AdminAccountState): Promise<MfaOutcome> {
  const client = asUntyped(getSupabaseServerClient());

  try {
    const { data: aalData } = await client.auth.mfa.getAuthenticatorAssuranceLevel();
    const { currentLevel, nextLevel } = aalData ?? {};

    if (nextLevel === 'aal2' && currentLevel !== 'aal2') {
      return { kind: 'step_up_required' };
    }

    // No factors enrolled (nextLevel !== 'aal2').
    const deadline = state.mfaGraceDeadline;
    const withinGrace = deadline !== null && deadline.getTime() > Date.now();

    if (!withinGrace) {
      return { kind: 'grace_expired' };
    }
```

```
    return { kind: 'ok', requiresMfaSetup: true, mfaGraceDeadline: deadline };
  } catch (err) {
    console.error('[Admin] evaluateAdminMfa error — failing closed, requiring step-up', err);
    return { kind: 'step_up_required' };
  }
}
```

`currentLevel`/`nextLevel` come directly from Supabase's own `auth.mfa.getAuthenticatorAssuranceLevel()` — `nextLevel` reflects what the *account's enrolled factors* require, and `currentLevel` reflects what the *current session* has actually verified. The `nextLevel === 'aal2' && currentLevel !== 'aal2'` check is exactly Supabase's documented pattern for "this session needs to complete a second factor before it can be treated as trusted," so this is step-up (AAL2) enforcement, not merely factor enrollment.

2.1 Four call sites, one policy, applied by surface type

```
export async function requireAdmin(): Promise<{
  user: User;
  requiresMfaSetup: boolean;
  mfaGraceDeadline: Date | null;
}> {
  const client = asUntyped(getSupabaseServerClient());
  const { data: { user } } = await client.auth.getUser();

  if (!user) {
    redirect('/auth/sign-in');
  }

  const state = await getAdminAccountState(user.id);
  if (!state.isAdminFlag) {
    redirect('/');
  }

  const outcome = await evaluateAdminMfa(state);

  if (outcome.kind === 'step_up_required') {
    console.log('[Admin] session is aal1 but aal2 factors enrolled — redirecting to /auth/verify');
    redirect('/auth/verify');
  }

  if (outcome.kind === 'grace_expired') {
    console.log('[Admin] MFA grace period expired with no factors enrolled — forcing enrollment', {
      userId: user.id,
    });
    redirect('/settings?require_mfa=1#security');
  }

  return { user, requiresMfaSetup: outcome.requiresMfaSetup, mfaGraceDeadline: outcome.mfaGraceDeadline };
}

export async function requireAdminApi(): Promise<
  { user: User; requiresMfaSetup: boolean; mfaGraceDeadline: Date | null } | NextResponse
> {
```

```
  ...
  if (outcome.kind === 'step_up_required') {
    console.log('[Admin] API route blocked — aal2 required but session is aal1');
    return NextResponse.json(
      { error: 'MFA verification required. Please complete step-up authentication.' },
      { status: 403 },
    );
  }

  if (outcome.kind === 'grace_expired') {
    console.log('[Admin] API route blocked — MFA grace period expired with no factors enrolled');
    return NextResponse.json(
      { error: 'MFA enrollment required. Enable two-factor authentication in Settings to continue.' },
      { status: 403 },
    );
  }
  ...
}
```

1 `requireAdmin()` — Server Components / pages: redirects to `/auth/verify` (step-up) or `/settings?require_mfa=1#security` (enrollment required).

1 `requireAdminApi()` — `/api/admin/*` route handlers: returns `403` with an explicit MFA-related error message instead of redirecting (there's no browser navigation to redirect in an API response).

1 `requireAdminUser()` — Server Actions: throws `Error('MFA step-up required')` / `Error('MFA enrollment required')`, caught by the calling action and surfaced to the UI.

1 `requireAdminUserId()` — Server Actions that only need the caller's id: returns `null` (treated as "not authorized") under the same two conditions.

All four resolve to the exact same `evaluateAdminMfa()` outcome for the exact same account state — the only difference is *how* the "blocked" result is communicated back to the caller (redirect vs. JSON vs. thrown error vs. `null`), which is what each surface type requires.

---

3. Every admin surface is gated — no exceptions

3.1 Pages: layout-level enforcement makes bypass structurally impossible

`/admin`'s `layout.tsx` calls `requireAdmin()` **before rendering any

child route**. In Next.js App Router, every nested page under

`/admin/**` is composed inside this layout — there is no routing path

that reaches a page's content without first executing this call:

```
import { requireAdmin } from '~/lib/admin';
import { AdminSidebar } from './_components/admin-sidebar';
import { AdminMfaSetupBanner } from './_components/admin-mfa-setup-banner';
import { adminMainClass, adminShellBg } from './_components/admin-dash-tokens';

export default async function AdminLayout({ children }: { children: React.ReactNode }) {
  // Enforce authentication, admin status, and MFA assurance at the layout level.
  // Pages that also call requireAdmin() will benefit from the cached Supabase session.
  const { requiresMfaSetup, mfaGraceDeadline } = await requireAdmin();

  return (
    <div className={`flex min-h-screen flex-col md:flex-row ${adminShellBg}`}>
      <AdminSidebar />
      <div className="flex flex-1 flex-col">
        {requiresMfaSetup && <AdminMfaSetupBanner graceDeadline={mfaGraceDeadline} />}
        <main className={adminMainClass}>{children}</main>
      </div>
    </div>
  );
}
```

Every one of the 16 distinct admin pages in the codebase

(`/admin`, `/admin/users`, `/admin/audio`, `/admin/queued-artworks`,

`/admin/contacts`, `/admin/blog`, `/admin/blog/new`,

`/admin/blog/[id]/edit`, `/admin/api-keys`, `/admin/feedback`,

`/admin/emails`, `/admin/about`, `/admin/taco`, `/admin/leads`,

`/admin/pitch`) is a descendant of this single layout, and the

majority additionally call `requireAdmin()` again at the top of the

page itself (as shown for `/admin/page.tsx` above) — belt-and-suspenders

at the individual-page level on top of the structural layout guard.

3.2 `/api/admin/*` routes

Every route handler under `/api/admin/**` that performs an admin

action calls `requireAdminApi()` as its first statement:

```
import { requireAdminApi } from '~/lib/admin';
...
export async function GET(request: NextRequest) {
  const gate = await requireAdminApi();
  if (gate instanceof NextResponse) return gate;
  ...

import { requireAdminApi } from '~/lib/admin';
```

```
...
export async function POST(request: NextRequest) {
  const auth = await requireAdminApi();
  if (auth instanceof NextResponse) return auth;
  ...
```

The one exception, `/api/admin/check/route.ts`, is intentionally not gated the same way because it is not an administrative *action* — it is the client-side "should I show the admin nav link" status probe, implemented with the MFA-agnostic `isAdmin()` boolean helper and returning only `{ isAdmin: boolean }`. It performs no privileged read or write and leaks no data beyond what the caller's own session already reveals about itself, so it is out of scope for MFA enforcement by design (there is nothing here for MFA to protect).

3.3 Server Actions

Every admin Server Action that mutates data or returns non-public/administrative data calls `requireAdminUser()` (or `requireAdminUserId()`) as its first step, e.g.:

```
'use server';

import { requireAdminUser } from '~/lib/admin';
```

This pattern is consistent across `admin-feedback.ts`, `link-artworks-to-exhibition.ts`, `lead-invite-outreach.ts`, `manage-featured-artworks.ts`, `revoke-free-access.ts`, `grant-free-access.ts`, `api-keys-admin.ts`, `search-user-for-admin.ts`, `admin-contacts.ts`, `email-templates-admin.ts`, `about-content.ts`, and `blog-posts-admin.ts` — every Server Action under `/admin` that touches non-public data or performs a mutation. (Three read-only helpers — `getFeaturedEntry()`, `getFeaturedArtworksList()`, and `getQueuedArtworks()` — are intentionally excluded from this list: they return only already public, `status = 'verified' AND is_public = true` artwork columns. `getFeaturedEntry()` is in fact also called directly from the public

homepage (`src/app/page.tsx`) and `src/app/v1/page.tsx`, confirming by usage that this data was never intended to be admin-gated in the first place — there is no administrative privilege being exercised by these three functions for MFA to protect.)

---

4. Why "no MFA enrolled" doesn't mean "no enforcement": the grace period is itself enforced

A newly-promoted admin cannot have already completed TOTP enrollment at the instant they're granted the role, so an admin interface that hard-blocked immediately would risk locking out the only administrator before they have a chance to set up a factor. Provenance resolves this without ever leaving MFA unenforced indefinitely: a **database trigger**, not application code, starts a 7-day countdown the moment an account is granted admin — so the deadline cannot be forgotten, skipped, or manipulated by adjusting request parameters:

```
alter table public.accounts
  add column if not exists admin_mfa_grace_deadline timestamptz;

comment on column public.accounts.admin_mfa_grace_deadline is
  'Deadline by which an admin must enroll MFA before requireAdmin()/requireAdminApi() hard-block them. Set automat

create or replace function kit.set_admin_mfa_grace_deadline()
returns trigger
language plpgsql
security definer
set search_path = ''
as $$
begin
  -- Admin newly granted (false/null -> true): start a 7-day MFA enrollment
  -- grace period. Runs regardless of role — this is bookkeeping, not the
  -- privilege-escalation guard (that stays in kit.protect_admin_flag()).
  if (new.public_data ->> 'admin')::boolean is true
     and coalesce((old.public_data ->> 'admin')::boolean, false) is false then
    new.admin_mfa_grace_deadline := now() + interval '7 days';
  end if;

  -- Admin revoked: clear any stale deadline so it doesn't linger if the
  -- account is re-granted admin later without this trigger re-running logic
  -- being relied upon.
  if coalesce((new.public_data ->> 'admin')::boolean, false) is false
     and (old.public_data ->> 'admin')::boolean is true then
    new.admin_mfa_grace_deadline := null;
  end if;

  return new;
end;
```

```
$$;

create trigger set_admin_mfa_grace_deadline_trigger
  before update on public.accounts
  for each row
  execute function kit.set_admin_mfa_grace_deadline();
```

The same migration backfilled every *existing* admin account with a

fresh 7-day deadline at deploy time, closing the gap for accounts that

predated this control rather than grandfathering them in indefinitely:

```
-- Backfill: any account that is *already* admin today gets a fresh 7-day
-- grace window starting now, rather than being retroactively hard-blocked
-- (or perpetually grandfathered in with no deadline at all) the moment this
-- migration ships.
update public.accounts
set admin_mfa_grace_deadline = now() + interval '7 days'
where (public_data ->> 'admin')::boolean is true
  and admin_mfa_grace_deadline is null;
```

`evaluateAdminMfa()` treats a missing/null deadline as **already

expired** (`withinGrace` is only `true` when a deadline exists *and*

is in the future — see §2), so a bug or data anomaly that leaves this

column unset fails closed to "MFA enrollment required," never to

"MFA not required."

During the grace window, the admin shell surfaces a persistent,

increasingly urgent reminder rather than silence:

```
export function AdminMfaSetupBanner({ graceDeadline }: { graceDeadline?: Date | null }) {
  const daysLeft = daysUntil(graceDeadline);
  const urgent = daysLeft !== null && daysLeft <= 2;
  ...
  <strong>Security notice:</strong> Your admin account does not have two-factor
  authentication enabled.{' '}
  {daysLeft !== null
    ? daysLeft === 0
      ? 'You must enable MFA today to keep admin access.'
      : `You have ${daysLeft} day${daysLeft === 1 ? '' : 's'} left to enable MFA before admin access is blocked.`
    : 'Enable MFA to protect admin access.'}{' '}
  <Link href="/settings#security">Set up MFA in Settings →</Link>
```

---

5. Enrollment and step-up UI

Enrollment happens in Settings → Security

(`src/app/settings/_components/security-section.tsx`), which renders

`MultiFactorAuthFactorsList` (TOTP/authenticator-app enrollment) and

explicitly explains *why* the user landed there when arriving via a

blocked-admin redirect:

```
export function SecuritySection({ userId, email, mfaEnrollmentRequired }: Props) {
  ...
  {mfaEnrollmentRequired && (
    <div role="alert" ...>
      <strong>Admin access blocked:</strong> your grace period to enable
      two-factor authentication has expired. Enroll an authenticator app
      below to regain access to the admin dashboard.
    </div>
  )}
```

Step-up verification (for admins who already have a factor

enrolled but whose current session hasn't completed it) happens at

`/auth/verify`, gated by the same underlying AAL check via

`checkRequiresMultiFactorAuthentication()`:

```
async function VerifyPage(props: Props) {
  const client = getSupabaseServerClient();
  const { data } = await client.auth.getClaims();

  if (!data?.claims) {
    redirect(pathsConfig.auth.signIn);
  }

  const needsMfa = await checkRequiresMultiFactorAuthentication(client);

  if (!needsMfa) {
    redirect(pathsConfig.auth.signIn);
  }
  ...

export async function checkRequiresMultiFactorAuthentication(
  client: SupabaseClient,
) {
  const assuranceLevel = await client.auth.mfa.getAuthenticatorAssuranceLevel();
  ...
  const { nextLevel, currentLevel } = assuranceLevel.data;
  return nextLevel === ASSURANCE_LEVEL_2 && nextLevel !== currentLevel;
}
```

---

6. Conclusion

Multi-factor authentication for administrative access is enforced by

a single shared server-side check (`evaluateAdminMfa()` in

`src/lib/admin.ts`), invoked by every admin page (structurally, via

the shared `/admin` layout, with individual pages also re-checking),

every `/api/admin/*` route, and every admin-privileged Server Action.

The only excluded endpoints are ones that expose no administrative

privilege at all (an admin-status probe returning a boolean, and

read-only accessors for data that is already public elsewhere in the app) — there is no administrative capability reachable without first satisfying this check. Enforcement for accounts with no enrolled factor is time-bounded by a database-trigger-managed, fail-closed 7-day grace deadline rather than being left open indefinitely, and accounts with an enrolled factor are hard-blocked from admin access until their session completes AAL2 step-up.